

PART 3

安全に使うために

目標：使う・使わないを、理由をつけて判断できるようになる



長所と短所を比べて、判断する

長所

- 習得が簡単: コア部分は難しくない
- 使いやすい: Instance を書くだけ
- 速い: GetComponent や Find が不要

短所

- 依存が隠れる: バグの原因を追いにくい
- テストが難しい: 状態が全体に影響する
- 密結合: リファクタリングが難しい

長く運用する大規模なゲームでは**避ける**。小規模なゲームでは魅力的。
使うなら**最小限**に。グローバルアクセスが本当に必要な一握りだけに。

4つの質問がすべて YES なら検討する

4つの質問

- ① 本当にゲーム全体で**1つ**だけ？（将来も？）
- ② 本当にあちこちから必要？
- ③ シーンをまたいで生き続ける必要がある？
- ④ 他の方法で済まない？

候補	判断	理由
AudioManager	使う	BGMは1つ・全体から
SaveManager	使う	保存先は1つ
GameManager	慎重に	肥大化しやすい
Player	使わない	2人プレイで破綻
Enemy	使わない	たくさんいる
ScoreUI	使わない	シーンごとの表示

シングルトン 6つの約束

1. 数を絞る

目安はプロジェクトで2~3個まで

2. 外から直接書き換えさせない

`private set` / メソッド経由で変更

3. Awake では自分の準備だけ

他の Instance に触るのは Start 以降

4. シーン内の参照を持たない

`DontDestroyOnLoad` するのは特に

5. 終了処理では触らない

`OnDestroy` / `OnApplicationQuit`

6. リセット方法を用意する

`ResetGame()` などを明示的に呼ぶ

書き換えの入口を1つにする

C# GameManager.cs

```
1 public class GameManager : Singleton<GameManager>
2 {
3     // 読むのは誰でも、書くのは自分だけ
4     public int Score { get; private set; }
5
6     public void AddScore(int value)
7     {
8         if (value <= 0) return;    // おかしな値をはじく
9         Score += value;
10    }
11
12    public void ResetGame()
13    {
14        Score = 0;
15    }
16 }
```

危険①への対策

- 外から **Score = 0** と書くとコンパイルエラー
- 変更の入口が **AddScore** だけ → ブレークポイント1か所で犯人が分かる

「どこからでも」が本当に必要か考える

方法	向いている場面
[SerializeField] で参照を渡す	同じシーン内。依存が Inspector に見える
生成する側から渡す	Instantiate したオブジェクト（Spawner が渡す）
イベント（オブザーバー）	「起きたこと」を知らせるだけでよいとき（今後の授業）
ScriptableObject	複数のオブジェクトで共有したい設定やデータ
static クラス	状態を持たない計算（Mathf のようなもの）

POINT

E-Book でも、オブジェクトプールやイベントマネージャーをシングルトンにする例が紹介されている。他のパターンと組み合わせることも多い。

HOMEWORK

Unityで実践

2種類のシングルトンを作り、Lib に入れる

4

2種類のシングルトンを Lib に入れる

- **Assets/Lib** フォルダに、次の2つのスクリプトを作る（配布コードの模写でOK）
- 今後のデザインパターンの授業では、この **Lib** のシングルトンを使っていく

SceneSingleton<T>

シーンを切り替えると一緒に破棄される。
シーン内の UI などを参照してよい。
例: StageManager、ステージの UI 管理

PersistentSingleton<T>

DontDestroyOnLoad でシーンをまたいで残る。
シーン内オブジェクトの参照は持たない。
例: GameManager、AudioManager

POINT

どちらも危険④⑤⑥⑧の対策入り。Title → Game → Title と移動し、Hierarchy でそれぞれがどう残るか・消えるかを確認すること。

SceneSingleton<T> (シーンごとに破棄)

C# Assets/Lib/SceneSingleton.cs

```
1 using UnityEngine;
2
3 // シーンを切り替えと一緒に破棄されるシングルトン
4 public class SceneSingleton<T> : MonoBehaviour where T : Component
5 {
6     private static T instance;
7
8     public static T Instance
9     {
10         get
11         {
12             // 見つからなくても新しくは作らない (null が返る)
13             if (instance == null)
14             {
15                 instance = FindAnyObjectByType<T>();
16             }
17             return instance;
18         }
19     }
20 }
```

ポイント

- 自動で作らない → Inspector の設定がない空の物が生まれない
- Awake 前に読まれても Find で見つける (危険④)

SceneSingleton<T> (続き)

C# Assets/Lib/SceneSingleton.cs

```
21     protected virtual void Awake()
22     {
23         if (instance == null) { instance = this as T; }
24         else if (instance != this) { Destroy(gameObject); } // 危険⑤
25     }
26
27     protected virtual void OnDestroy()
28     {
29         // シーン破棄で自分が消えたら、登録を外す
30         if (instance == this) { instance = null; }
31     }
32 }
```

ポイント

- **instance != this** で判定
(危険⑤)
- **OnDestroy** で登録を外す
→ 次のシーンで新しい物が登録される

PersistentSingleton<T> (シーンをまたいで残る)

C# Assets/Lib/PersistentSingleton.cs

```
1 using UnityEngine;
2
3 // シーンを切り替えても破棄されないシングルトン
4 public class PersistentSingleton<T> : MonoBehaviour where T : Component
5 {
6     private static T instance;
7     private static bool isQuitting;
8
9     public static T Instance
10    {
11        get
12        {
13            if (isQuitting) { return null; } // 終了中は作らない
14            if (instance == null) { SetupInstance(); }
15            return instance;
16        }
17    }
18
```

ポイント

- こちらはなければ自動で作る (遅延初期化)
- ゲーム終了中は作らない
→ 幽霊を防ぐ (危険⑧)

PersistentSingleton<T> (続き)

C# Assets/Lib/PersistentSingleton.cs

```
19     private static void SetupInstance()
20     {
21         instance = FindAnyObjectByType<T>();
22         if (instance == null)
23         {
24             GameObject gameObj = new GameObject(typeof(T).Name);
25             instance = gameObj.AddComponent<T>();
26         }
27     }
28
29     protected virtual void Awake()
30     {
31         if (instance == null) { instance = this as T; }
32         else if (instance != this)
33         {
34             Destroy(gameObject); // 危険⑤対策
35             return;
36         }
37         transform.SetParent(null); // ルートでないと効かない
38         DontDestroyOnLoad(gameObject);
39     }
```

ポイント

- AddComponent した瞬間に Awake が動く
- → DontDestroyOnLoad は **Awake** に一本化
- 親から外してから DontDestroyOnLoad

PersistentSingleton<T> (続き)

C# Assets/Lib/PersistentSingleton.cs

```
40
41     protected virtual void OnApplicationQuit()
42     {
43         isQuitting = true;
44     }
45
46     protected virtual void OnDestroy()
47     {
48         if (instance == this) { instance = null; }
49     }
50 }
```

ポイント

- 終了の合図を受けたら
isQuitting = true
- 以降は Instance が null を返す → 使う側は null チェック

継承して使い分ける

C# 使用例

```

1  // シーンをまたいで残る
2  public class GameManager : PersistentSingleton<GameManager>
3  {
4      public int Score { get; private set; }
5      public void AddScore(int value) { if (value > 0) Score += value; }
6      public void ResetGame() { Score = 0; }
7  }
8
9  // シーンごとに置く (UI の参照を持ってよい)
10 public class StageManager : SceneSingleton<StageManager>
11 {
12     [SerializeField] private TMP_Text scoreText;
13
14     protected override void Awake()
15     {
16         base.Awake();    // 必ず呼ぶ
17     }
18 }

```

使い分け

- シーンをまたぐ? → **Persistent**
- シーン内の物を参照する? → **Scene**
- Awake を書くなら **base.Awake()**

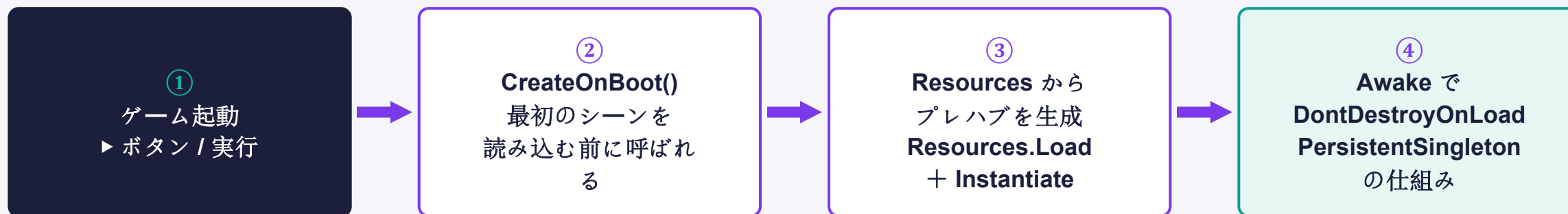
PRACTICE

SoundManager を作る

起動時に Resources のプレハブから自動生成し、サウンドを再生する



シーンに置かなくても、起動時に必ず1つ生まれる



- どのシーンから再生しても **SoundManager** が存在する（テスト用シーンでも音が鳴る）
- **AudioSource** や音のファイルはプレハブに設定しておく → 遅延初期化の「空っぽの物」問題が起きない

POINT

使うのは **RuntimeInitializeOnLoadMethod**（起動時に呼ばれる）と **Resources.Load**（プレハブを読み込む）の2つ。

フィールドと、起動時の自動生成

```
C# Assets/Scripts/SoundManager.cs
1  using UnityEngine;
2
3  public class SoundManager : PersistentSingleton<SoundManager>
4  {
5      [SerializeField] private AudioSource bgmSource;
6      [SerializeField] private AudioSource seSource;
7      [SerializeField] private AudioClip[] bgmClips;
8      [SerializeField] private AudioClip[] seClips;
9
10     // ゲーム起動時（最初のシーンを読み込む前）に1回だけ呼ばれる
11     [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
12     private static void CreateOnBoot()
13     {
14         SoundManager prefab = Resources.Load<SoundManager>("SoundManager"); // Resources/SoundManager.prefab
15         if (prefab == null)
16         {
17             Debug.LogError("Resources に SoundManager プレハブがありません");
18             return;
19         }
20         Instantiate(prefab); // → Awake で DontDestroyOnLoad される
21     }
```

起動時に生成する2つの道具

RuntimeInitializeOnLoadMethod

- **static** メソッドに付ける（インスタンスがまだ無いので）
- **BeforeSceneLoad**: 最初のシーンの **Awake** より前
- **AfterSceneLoad** にすると、シーンの **Awake / Start** の後になる
- エディタではどのシーンから ▶ しても呼ばれる

Resources.Load<T>()

- **Assets/Resources** フォルダの中を読み込む
- パスは **Resources** からの相対・拡張子なし 例:
"Sound/SoundManager"
- <SoundManager> と書くと、プレハブに付いたコンポーネントが返る
- 見つからなければ **null**

POINT

Instantiate した瞬間に PersistentSingleton の Awake が動き、**instance** 登録と **DontDestroyOnLoad** まで自動で済む。

BGM と SE を再生する

C# Assets/Scripts/SoundManager.cs

```
22
23     public void PlayBGM(string clipName)
24     {
25         AudioClip clip = FindClip(bgmClips, clipName);
26         if (clip == null) { return; }
27         bgmSource.clip = clip;
28         bgmSource.loop = true;
29         bgmSource.Play();
30     }
31
32     public void PlaySE(string clipName)
33     {
34         AudioClip clip = FindClip(seClips, clipName);
35         if (clip == null) { return; }
36         seSource.PlayOneShot(clip);
37     }
38
39     public void StopBGM() { bgmSource.Stop(); }
```

ポイント

- BGM は **clip** を差し替えてループ再生
- SE は **PlayOneShot** → 重ねて鳴らせる

名前で音を探す

C# Assets/Scripts/SoundManager.cs

```
40
41     private AudioClip FindClip(AudioClip[] clips, string clipName)
42     {
43         foreach (AudioClip clip in clips)
44         {
45             if (clip.name == clipName) { return clip; }
46         }
47         Debug.LogWarning(clipName + " という音が登録されていません");
48         return null;
49     }
50 }
```

ポイント

- clip.name = 音声ファイルの名前
- 見つからないときは警告を出して **null**
- 外からは使わないので **private**

SoundManager プレハブを作る

- ① Assets に **Resources** フォルダを作る（名前は必ず Resources）
- ② Hierarchy に空の GameObject「**SoundManager**」を作り、SoundManager.cs を付ける
- ③ 子に「BGM」「SE」を作り、それぞれ **AudioSource** を付ける（Play On Awake はオフ）
- ④ SoundManager の Inspector に、2つの AudioSource と、BGM・SE の音声ファイルを登録
- ⑤ Resources フォルダへドラッグしてプレハブ化し、Hierarchy からは削除する

注意

プレハブ名と Resources.Load("**SoundManager**") の文字列は完全に一致させる。シーンには置かない。

試してみる

どのシーンのどのスクリプトからでも鳴らせる

C# 使用例

```
1 // TitleScene に置いたスクリプト
2 private void Start()
3 {
4     SoundManager.Instance.PlayBGM("Title");
5 }
6
7 // Coin.cs (コインを取ったら)
8 private void OnTriggerEnter2D(Collider2D other)
9 {
10     SoundManager.Instance.PlaySE("Coin");
11     Destroy(gameObject);
12 }
```

確認すること

- どのシーンから ▶ しても Hierarchy の **DontDestroyOnLoad** に SoundManager がある
- シーンを切り替えても **BGM** が途切れない
- "Title" "Coin" は登録した音声ファイル名

注意

SoundManager で気をつけること

シーンに置かない

起動時に生成済みなので、シーンに置いた物は重複として消される（危険⑥と同じ）。設定はプレハブで行う。

Resources の中身

Resources に入れた物は使っていなくても全部ビルドに含まれる。使う物だけ入れる。大規模なら Addressables。

終了時は null

PersistentSingleton は終了中 **null** を返す。OnDestroy で鳴らすなら Instance?.PlaySE(...)

POINT

SoundManager は「1つだけ・どこからでも・シーンをまたぐ」の4つの質問すべてに **YES** になる、シングルトンの代表例。